

Lab - 01

Abstract Class vs. Interface:

In java, Interface supports multiple inheritance, while abstract classes do not. A class can extend only one abstract class, but abstract it can implement multiple interfaces. This also avoids ambiguity problems that occur in multiple class inheritance.

An abstract class contains abstract methods, ~~not~~ concrete methods, instance variables and constructor. It is ~~not~~ suitable when classes share common state and behavior.

An Interface mainly defines method declarations and constants. It is preferred when different classes need to follow the same behavior but are not related by inheritance.

Lab - 02

Encapsulation is the process of ~~wa~~ wrapping data and method in a single unit and resisting direct access to the data using access modifiers like 'private'. This ensures data security by providing unauthorized access and data integrity by allowing validation before modification.

Here is the code for the bank account:

```
Class Bank Account {  
    Private String accNumber;  
    Private Double balance;  
  
    public void setAccountNumber(String AccNo) {  
        if (AccNo != Null && !AccNo.isEmpty()) {  
            this.accNumber = AccNo;  
        }  
    }  
}
```

}

```

public void setInitialBalance (double bal) {
    if (bal ≥ 0) {
        this.balance = bal;
    }
}
}

```

Lab - 03

Multi Threading Car parking

```

public class Register parking {
    Private String Car Num;
    Public Register parking (String CN) {
        this.Car Num = CN;
    }
    Public String getCarNumber () {
        return Car Num;
    }
}

```

```
import java.util.*
```

```
public class parkingPool {
```

```
    private Queue<RegisterParking> q = new LinkedList<>();
```

```
    public synchronized void addCar (RegisterParking car){
```

```
        q.add(car);
```

```
        notifyAll();
```

```
    }
```

```
    public synchronized RegisterParking getCar() throws  
        InterruptedException
```

```
    {
```

```
        while (q.isEmpty()){
```

```
            wait();
```

```
        }
```

```
        return q.poll();
```

```
    }
```

```
}
```

Parking Agent!

```
public class ParkingAgent extends Thread {
```

```
    private ParkingPool pool;
```

```
    private String AgentName;
```



```

public parkingAgent (ParkingPool pl, String nm) {
    this.pool = pl;
    this.Agentname = nm;
}

public void run() {
    try {
        while (true) {
            Register parking car = pool.getCar();
            System.out.println (Agentname + " Parked"
                                + car.get carNumber());
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}
}

```

main class:

```

public class MainClass {
    public static void main (String[] args) {

```

```
ParkingPool pool = new ParkingPool();
```

```
new ParkingAgent(pool, "Agent-1").Start();
```

```
new ParkingAgent(pool, "Agent-2").Start();
```

```
pool.addCar(new RegisteredParking("ABC123"));
```

```
pool.addCar(new RegisteredParking("XY2456"));
```

```
}
```

```
}
```

Lab-04

JDBC (Java Database Connectivity) enables Java applications to communicate with relational databases using standardized APIs. It hides database-specific details and provides uniform way to execute SQL queries.

Steps to execute a Select query :

- i) Load JDBC driver
- ii) Establish a connection to with database.
- iii) Create a Statement.
- iv) Execute the query.
- v) get the results in resultset.
- vi) close resources.

Code:

```
Try {
```

```
    Connection con = DriverManager.getConnection (
        url, user, pass);
```

```
    PreparedStatement ps = con.prepareStatement (
        "Select * from student");
```

```
    ResultSet rs = con.execute ps.executeQuery();
```

```
} Catch (Exception e) {
```

```
    e.printStackTrace();
```

```
} finally {
```

```
    con.close();
```

```
}
```

Lab-05

In Java EE, a servlet acts as a Controller in the MVC ~~architecture~~ architecture.

It handles user requests, processes business logic, and decides which view should display the response.

Example:

```
@WebServlet("/Student")
```

```
public class StudentServlet extends HttpServlet {  
    protected void doGet(HttpServletRequest req,  
                           HttpServletResponse res)  
        throws ServletException, IOException {
```

```
        req.setAttribute("name", "Shihab");
```

```
        RequestDispatcher rd = req.getRequestDispatcher("Student.jsp");
```

```
        rd.forward(req, res);
```

```
    }
```

```
}
```


Student.JSP :

<h2> Student ~~ID~~ Name : \${name} </h2>

Lab - 06

'Prepared - Statement' is preferred over 'Statement' because it provides better performance and security. It uses ~~Prepared Statement~~ SQL queries which reduces execution time when the same queries are executed multiple times.

It also uses parameterized queries, which prevent SQL injection attacks by ~~the~~ treating user input as data, not executable SQL.

Example:

```
PreparedStatement ps = Con. PreparedStatement (  
    'Insert into Students values (?, ?)');
```

```
ps.setInt(1, 100);
```

```
ps.setString(2, "Shirab");
```

```
ps.executeUpdate();
```

Lab-07

A ResultSet store data retrieved from a data base query.

next() → moves to the next row.

getInt() → retrieves Integer data.

getString() → retrieves String data.

Example:

```
ResultSet rs = ps.executeQuery();
```

```
while (rs.next()) {
```

```
    int id = rs.getInt("id");
```

```
    String name = rs.getString("name");
```

```
}
```

Lab-8

Spring Boot simplifies RESTful service development by providing auto-configuration, an embedded Server, and annotation based programming. which reduce boilerplate code. It also offers automatic JSON conversion, allowing developers to focus on business logic instead of configuration.

example!

~~public class~~

@RestController

public class StudentController {

 @GetMapping("/students")

 public List<Student> getStudents() {

 return repo.findAll();

 }

 @PostMapping("/students")

 public Student addStudent(@RequestBody Student s) {

 return ~~repo.Student~~ repo.save(s);

 }